

AUTOMATA

Syllabus of TOC

- ① Regular Language and Finite Automata (80%)
- ② Context free Language and push down Automata (10%)
- ③ REC, RE and Turing Machine (10%)

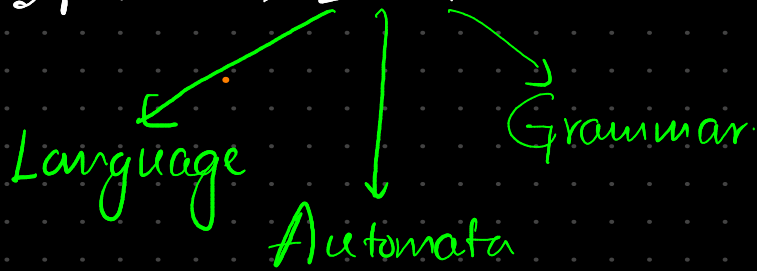
④ Closure properties and undecidability ★★

├ Language
├ Grammar
└ Automata (Machine)

├ Tables
├ language identify
└ Automata

TOC → concept → principles.

3 pillars → LAG



Symbol.

$\{a, b, c, \dots, 0, 1, 2, \dots\}$

String → Collection of alphabet, sequence.

Alphabet $\Sigma(a, b)$
Finite Set of symbol.

Language → Collection of strings.

$\Sigma(a, b)$ $\Rightarrow L_1 = \text{string of length 3}$

$\Rightarrow L_2 = \infty$

All the strings start with 'a' and end with 'a'.

$\{a a a, a a b, a b a, a b b, b a a, b a b, b b a, b b b\}$

$\{a, a a, a a a, a b b a, a b c b a, \dots, \infty\}$

Language length strings '0'
 $\Sigma^0 = \epsilon$

AUTOMATA

Language

→ FA
→ PDA
→ LBA
→ TM

Finite

Infinite

Automata (Model, Machine)

[whether a string is a part of language or not]

Mathematical Abstract Machine

POWER OF SIGMA

Power of Σ $\Sigma(a,b)$

Σ^0 = Set of all strings with length 0 = λ, ϵ

Σ^1 = Set of all strings with length 1 = $\{a, b\}$

Σ^2 = Set of all strings with length 2 = $\{aa, ab, ba, bb\}$

Σ^* = (Kleen closure) = ∞ Language
 $\hookrightarrow (a+b)^*$

$\Sigma^+ \rightarrow$ (positive closure) $\rightarrow$ all except ϵ

$$\Sigma^* = \Sigma^+ - \Sigma^0$$

$$\Sigma^* - \epsilon = \Sigma^+$$

GRAMMAR (G):

$\hookrightarrow$ defined as quadruple

$$G = \{V, T, P, S\}$$

Variable

Terminal

Production Rule

Start

$$S \rightarrow aSb \mid \epsilon$$

$\epsilon, aSb, aaSbb, aaaSbbb$

$\epsilon, ab, a^2Sb^2, a^3Sb^3$

a^2b^2, a^3b^3

a^nSb^n

$a^n b^n$

$n \geq 0$

$$S \rightarrow SS$$

$$S \rightarrow aSb$$

$$S \rightarrow bSa$$

$$S \rightarrow \epsilon$$

ϵ, aSb, bSa

$abba$

Language $\rightarrow$

$$n_a(w) = n_b(w)$$

DFA (Deterministic Finite Automata)

$DFA(Q, \Sigma, S, q_0, F)$

Set of Finite States

Set of alphabet (a, b)

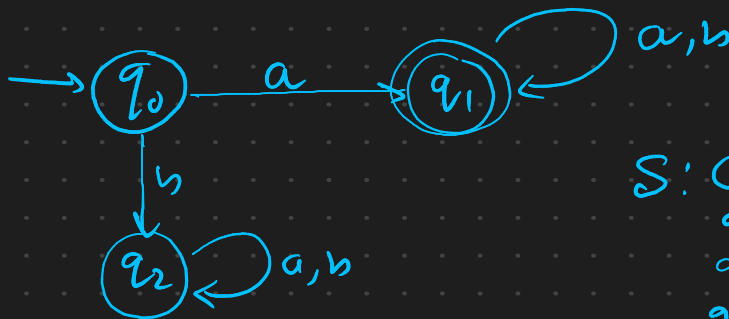
Transition

Start State

Set of finite final states

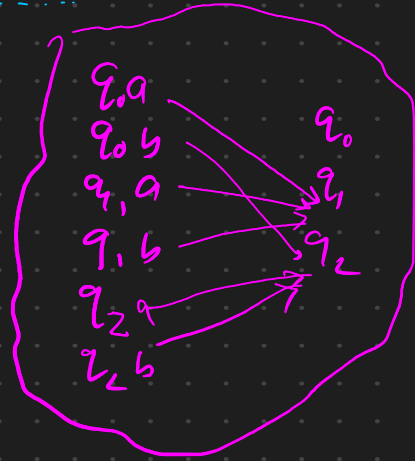
$$F \subseteq Q$$

String starting with 'a' $\rightarrow$ Language.
 $\{a, aa, aaa, ab, \underline{ba}, \dots\}$



$$S: Q \times \Sigma \rightarrow Q$$

q_0
 q_1
 q_2



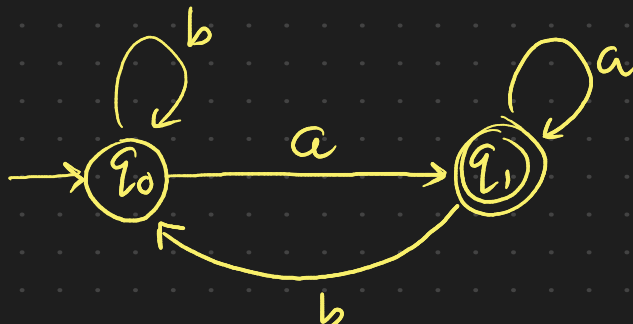
Q. Construct a DFA which accept a lang. of all strings containing 'a'

Ans: $\{a, aa, aba, ba, ab, \underline{b}, \dots\}$



Q. Construct a DFA which accept a language of all strings end with 'a'.

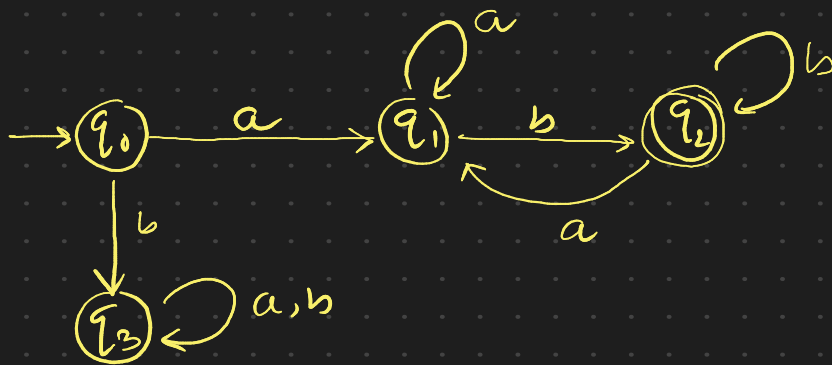
Ans: $L = \{a, aa, aaa, ba, baba, abba, baabba, \underline{ab}, abab, \dots\}$



Q. Construct a DFA which accept a language of all strings starting with 'a' and ending with 'b'.

Ans

$$L = \{ab, abab, ababab, \frac{ba}{x}, \frac{a}{x}, \frac{b}{x}, \dots, \infty\}$$



Q. Construct a DFA which accept a language of all strings not starting with 'a' or not ending with 'b'.

Ans

$$L = \{\lambda, \epsilon, \dots\}$$

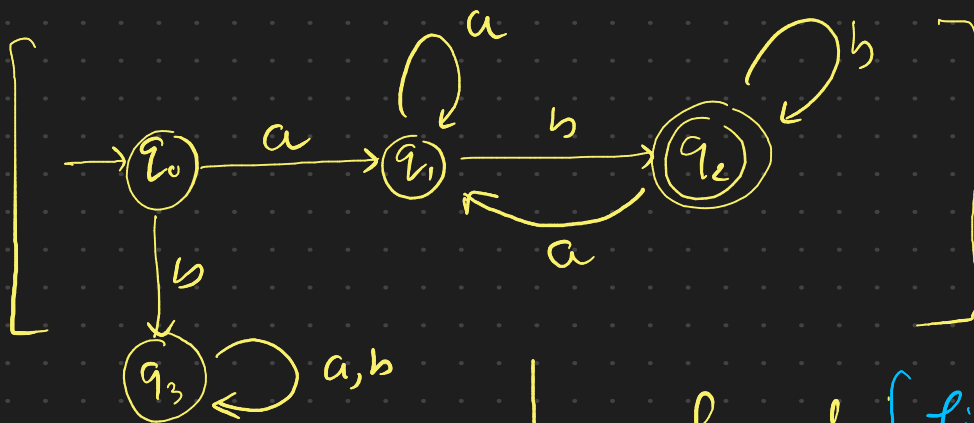
A or B

$$(A \cup B)^c = A^c \cap B^c \quad (A \cup B)$$

Starting with a

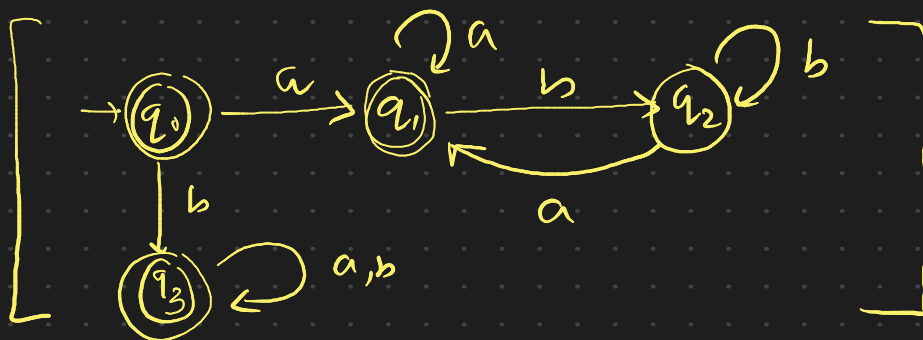
(8)

Ending with b



$(A \cup B)$

complement $\left[\begin{array}{l} \text{final} \rightarrow \text{non final} \\ \text{non final} \rightarrow \text{final} \end{array} \right]$



$(A \cup B)^c$

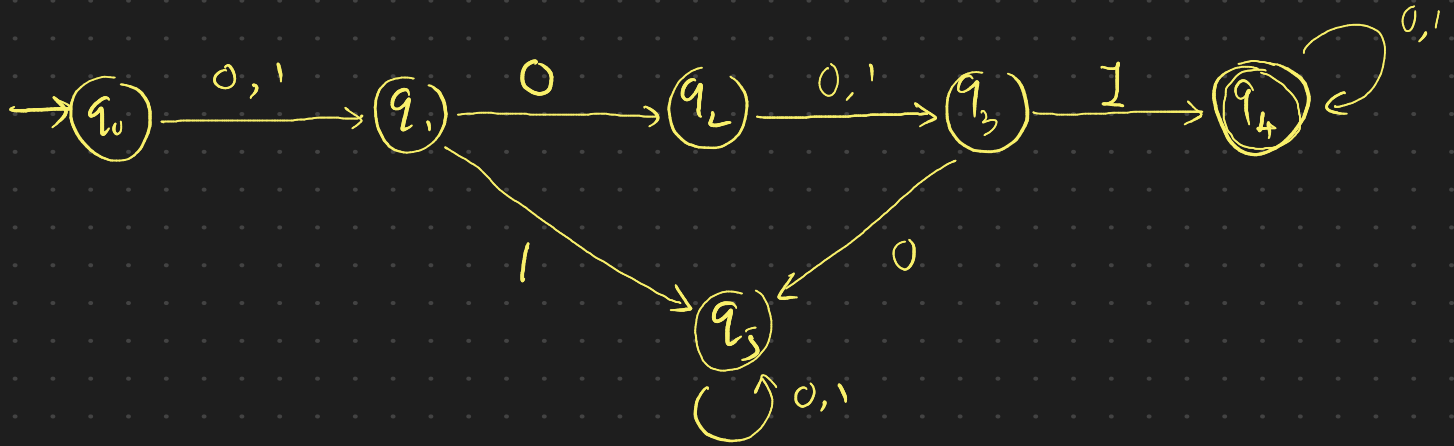
Q. Design a DFA which accepts all strings over (0,1) in which 2nd symbol is 0 and 4th is 1.

Ans.

0 1 0 1 (4)

min.

no. of states = $n+1 = 4+1 = 5$ ✓ (minimum length)



DFA

NDFA/NFA

① deal w/ not allowed

① allowed

② Multiple choices w/ attrib corresponding to an ip.

② available

③ ∈ not allowed

③ ∈ allowed

④ Digital computer

④ Non deterministic feature is not associated with real machines.

⑤ Designing & understanding difficult

⑤ Easy

⑥ No. of states ↑

⑥ ↓

Q.

NFA of all binary strings in which 2nd last bit is 1

$\Sigma(0,1)$

$$L = (1+0)^* \underline{1} (0+1)$$

11010, 01010, 11111

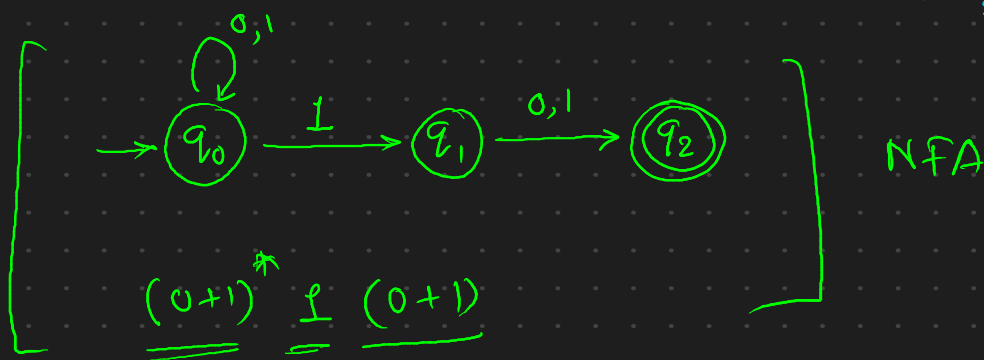
any comb of
1 and 0

2nd last symbol.

1/0 any one.

min string length = 2 : {10, 11} $\rightarrow n=2$

$\therefore \# \text{ states} = n+1 = 3$



	0	1
q_0	q_0	$q_0 q_1$
q_1	q_2	q_2
q_2	-	-

Transition table

	0	1
q_0	q_0	$q_0 q_1$
$q_0 q_1$	$q_0 q_2$	$q_0 q_1 q_2$
$q_0 q_2$	q_0	$q_0 q_1$
$q_0 q_1 q_2$	$q_0 q_2$	$q_0 q_1 q_2$



NFA

states (n)

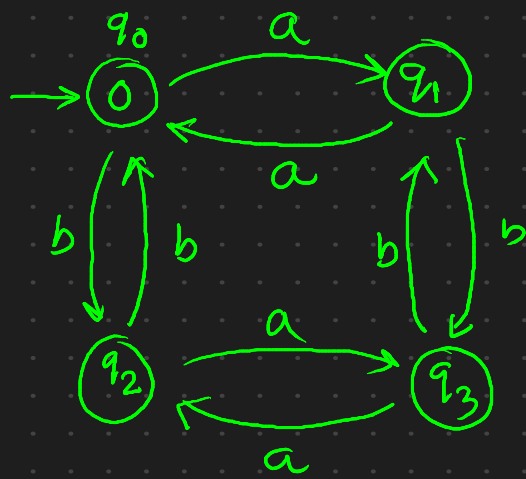
DFA

(2^n maximum)

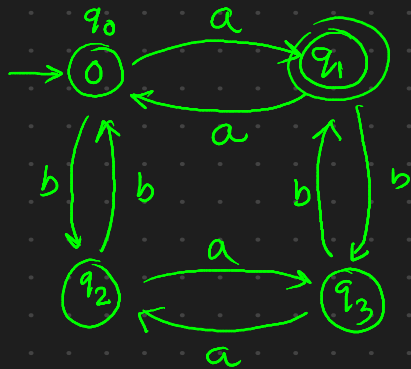
Let $L = \{ w \mid w \text{ has } \boxed{\text{even no. of a's}} \text{ and } \boxed{\text{even no. of b's}} \}$

$L = \{ \epsilon, aa, bb, \underline{ab}_x, abab, aabb, bbaa, baab, \dots \}$

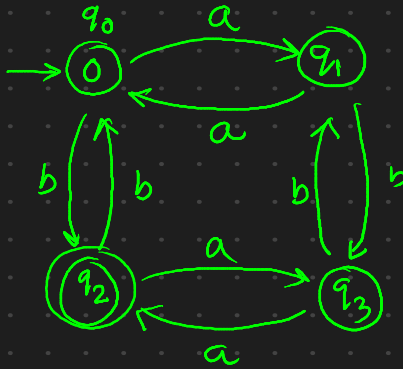
DEA



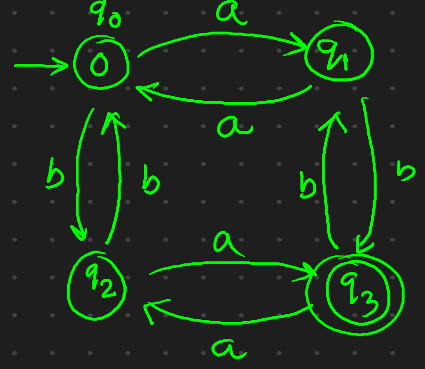
→ accepts even # b
and even # a



odd a, even b

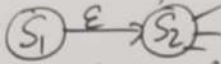


even a, odd b



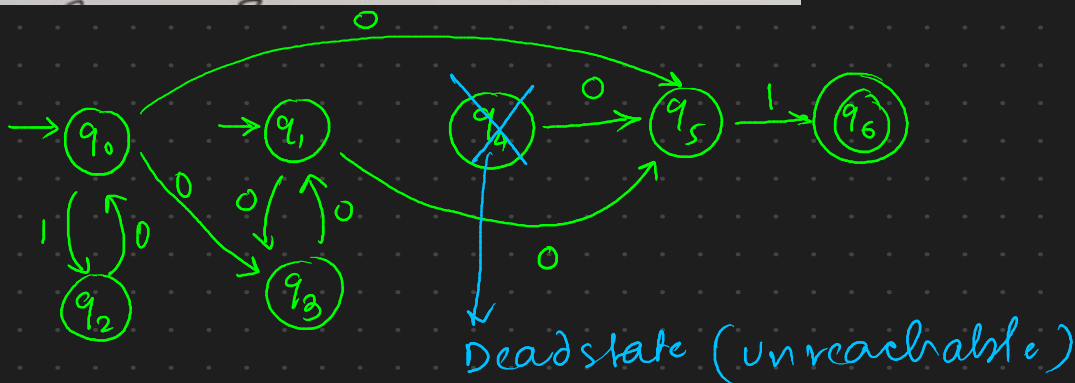
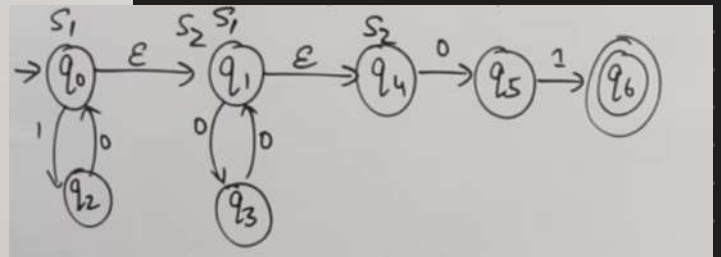
odd a, odd b

E-NFA (Epsilon NFA) → Eliminating E-Moves



- 1) find all edges starting from S_2
- 2) Duplicate all edges to S_1 without changing edge labels
- 3) if S_1 is initial state, make S_2 initial
- 4) if S_2 is final state, make S_1 final

5. Remove dead state.



Final NFA →



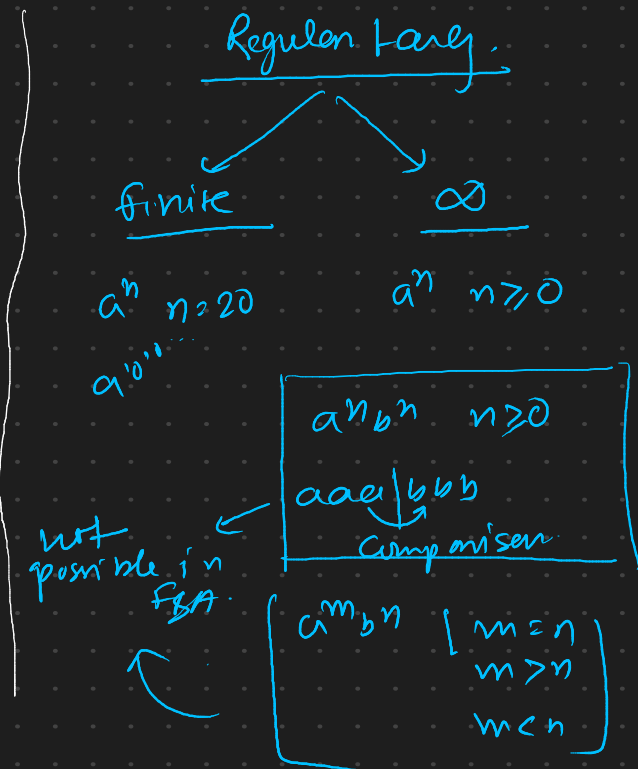
Limitations of FSA (Finite State Automata)

- ① Limited Memory
- ② Strings w/o comparison
- ③ Linear Power

a^{2^n} $a^i \rightarrow \text{print}$
 $\downarrow$
 XX

Application of FSA

- Word Processor program
- Digital Logic design
- Lexical Analyzer
- Switching Circuit Design
- Text Editors etc.
- Software for Scanning large bodies of text such as web pages to find occurrence of words, phrases, patterns.
- Software with finite states like vending Machines, Weigh Machine, Traffic lights, Toll Machine etc.
- Game design (Pac Man, Treasure Hunt etc.)

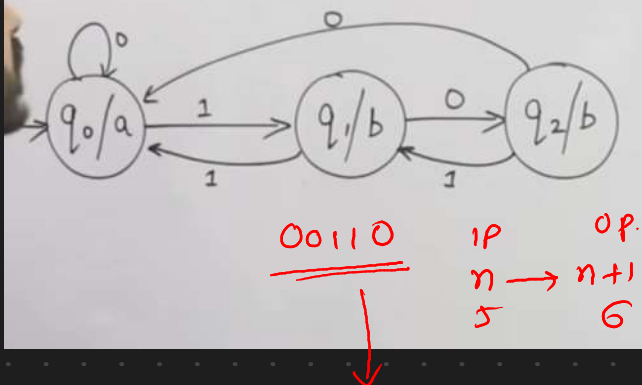


Moore Machine

Moore Machine

$$M_0 = \{Q, \Sigma, \Delta, \delta, \lambda, q_0\}$$

FA with output.



output: aaabaa
 ↓
 by default

Q = finite set of states $\{q_0, q_1, q_2\}$

Σ = input symbol (alphabet)

δ = Transition function
 $Q \times \Sigma$

q_0 = Start state (q_0)

Δ = Output symbol (a, b)

λ = Output function

$$\lambda: Q \rightarrow \Delta$$

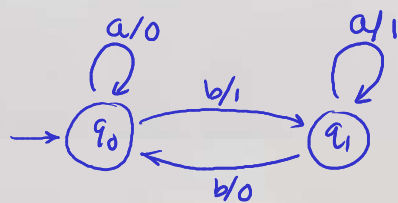
Transition table

	Current	Next	Op
	0	1	
q_0	q_0	q_1	a
q_1	q_2	q_0	b
q_2	q_0	q_1	b

Mealy Machine

Mealy Machine

$$M = \{Q, \Sigma, \Delta, \delta, \lambda, q_0\}$$



$q_0, a \rightarrow 0$
 $q_0, b \rightarrow 1$
 $q_1, a \rightarrow 1$
 $q_1, b \rightarrow 0$

Current State	Next State	
	if a	if b

Q = finite set of states
 Σ = input alphabet

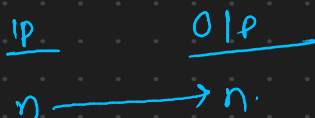
δ = Transition function
 $\delta: Q \times \Sigma \rightarrow Q$

q_0 = initial state

Δ = output symbol

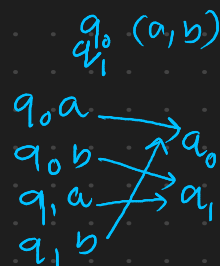
λ = Output function

$$\lambda: Q \times \Sigma \rightarrow \Delta$$



Transition Table

Current State	Next State			
	if a		if b	
	State	OP	State	OP
q_0	q_0	0	q_1	1
q_1	q_1	1	q_0	0

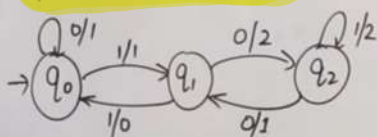


▲ Difference b/w Mealy & Moore Machine

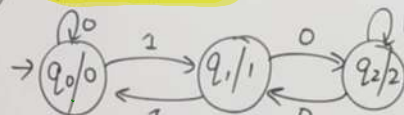
Mealy Machine

Moore Machine

- Output depends on present state and present input
- Input string of length 'n' → output length is also 'n'
- $\lambda: Q \times \Sigma \rightarrow \Delta$
- Mealy output is asynchronous



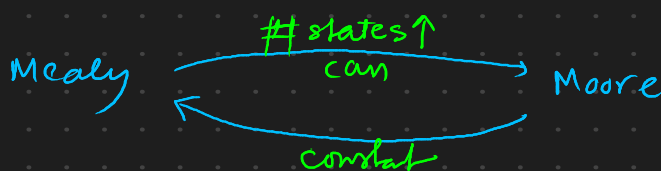
- Output only depends on present state
- Input string length 'n' → output string 'n+1'
- $\lambda: Q \rightarrow \Delta$
- Output is synchronous with clock



→ faster

→ slower comparative to Mealy

→ Conversion

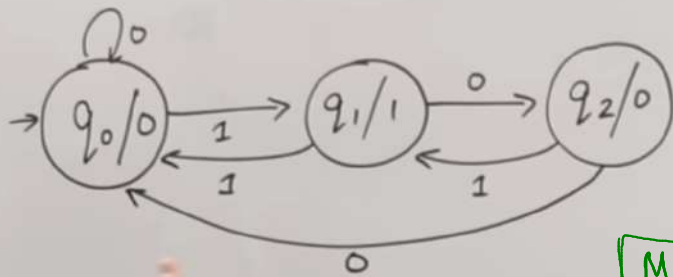


depend on both state & input

only depend on state

Moore $\rightarrow$ Mealy

Moore to Mealy Conversion

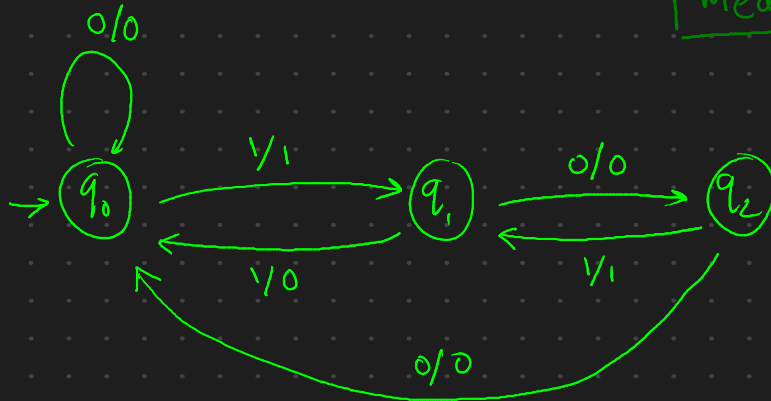


Moore

Mealy

Current States

	0	1	o/p
q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_0	q_1	0



Current State

	0	1
q_0	q_0 0	q_1 1
q_1	q_2 0	q_0 0
q_2	q_0 0	q_1 1

Moore

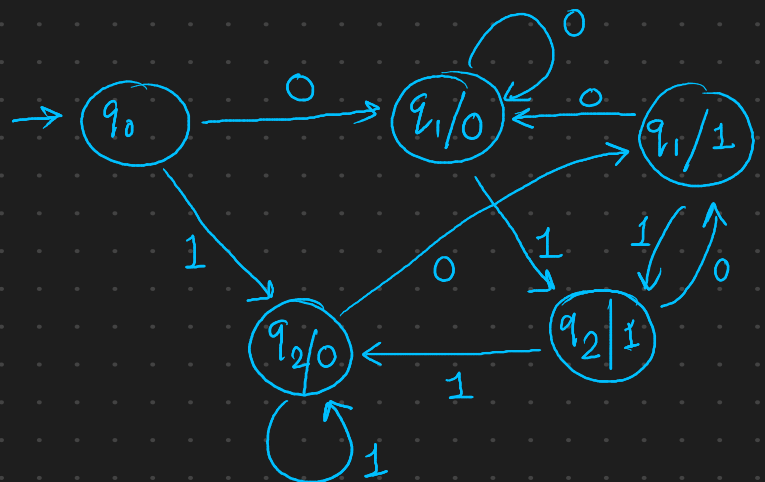
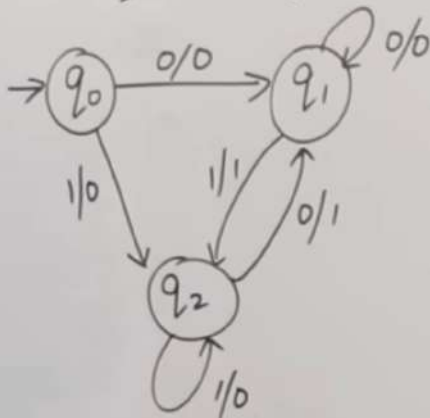
Mealy

m states
n output

m states

Mealy $\rightarrow$ Moore

Mealy to Moore Conversion



Mealy

Moore

m states
n outputs

m x n
States

(worst case)

3x2

6

E-NFA (Epsilon NFA)

Q Finite set of states

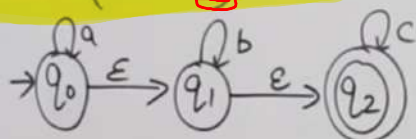
Σ input symbols 0,1

q_0 initial state

F Set of final states

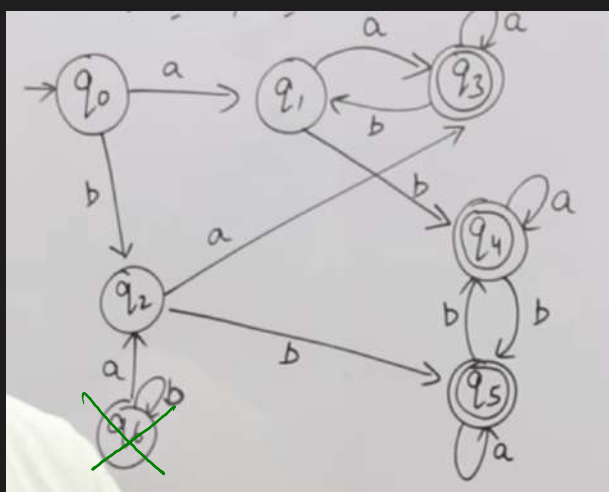
δ Transition function

$$\delta: Q \times (\Sigma \cup \epsilon) \rightarrow 2^Q$$



→ w/o giving any ip/symbol transition occur.

Minimization of DFA



	a	b
q0	q1	q2
q1	q3	q4
q2	q3	q5
q3	q3	q1
q4	q4	q5
q5	q5	q4

NFA → DFA

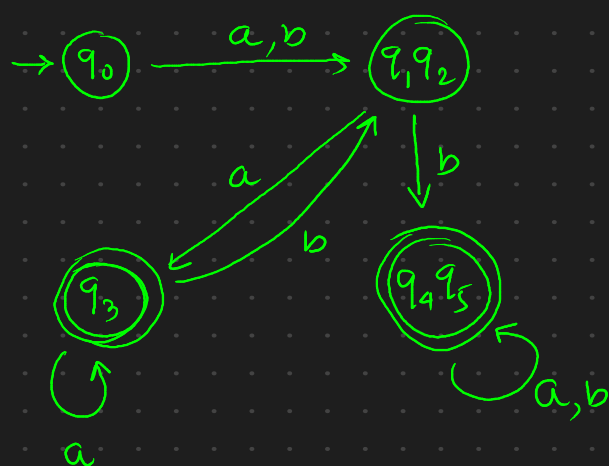
$$n \rightarrow 2^n$$

Steps

- 1) Delete unreachable state
- 2) convert states to classes

Accepting State
(Final)

Non Accepting State
(Non Final)



$$\pi_0 = \{q_0, q_1, q_2\} \quad G_1$$

$$\{q_3, q_4, q_5\} \quad G_2$$

$$\pi_1 = \{q_0\} \quad \{q_1, q_2\} \quad G_{11} \quad G_{12}$$

$$\{q_3\} \quad \{q_4, q_5\} \quad G_{21} \quad G_{22}$$

$$\pi_2 = \{q_0\} \quad \{q_1, q_2\}$$

$$\{q_3\} \quad \{q_4, q_5\}$$

4 states

Regular Expression $\rightarrow$ (Regular Languages) LA.

$\hookrightarrow$ Method to represent a language (L)

• Let R be a RE over alphabet Σ if R is -

① $\epsilon \rightarrow \text{set } \{\epsilon\}$

② $\phi \rightarrow \text{set } \{\}$

③ $a \in \Sigma \rightarrow \text{set } \{a\}$

④ $R_1 \cup R_2 = \text{Regular}$

⑤ $R_1 \cdot R_2 = \text{Regular}$

⑥ $(RE)^* = \text{Regular}$ (Kleen Closure)

Primitives. $\begin{cases} R = \epsilon : L(R) = \{\epsilon\} \\ R = \phi : L(R) = \{\} \\ R = a : L(R) = \{a\} \end{cases}$

Examples



① No string $\rightarrow \{\}$, $RE = \phi$

② length 0 $\rightarrow \{\epsilon\}$, $RE = \epsilon$

③ length 1 $\rightarrow \{a, b\}$, $RE = (a+b)$

④ length 2 $\rightarrow \{aa, ab, ba, bb\}$, $RE = (aa + bb + ab + ba) = (a+b)(a+b)$

⑤ length 3 $\rightarrow \{aaa, aab, abb, bbb, aba, \dots\}$, $RE = (a+b)(a+b)(a+b)$

⑥ Almost 1 $\rightarrow \{\epsilon, a, b\}$, $RE = (\epsilon + a + b)$

⑦ Almost 2 $\rightarrow \{\epsilon, a, b, aa, bb, ab, ba\}$, $RE = (\epsilon + a + b)(\epsilon + a + b)$

⑧ Almost 2 b's & 1 a's $\rightarrow \{\epsilon, abb, bba, a, b, baab\}$, $RE = (\epsilon + abb + bba + a + b + baab)$

Example - 2 (For ∞ Languages)

Consider Alphabet $\Sigma = \{a, b\}$

1) All strings having a single 'b' $\rightarrow$ ① a^*ba^*

2) All strings having at least one 'b' $\rightarrow$ ② $(a+b)^*b(a+b)^*$

3) " " " bbbb as substring $\rightarrow$ ③ $(a+b)^*bbbb(a+b)^*$

4) All strings end with 'ab' $\rightarrow$ ④ $(a+b)^*ab$

5) " " start with 'ba' $\rightarrow$ ⑤ $ba(a+b)^*$

6) " " beginning and end with 'a' $\rightarrow$ ⑥ $a(a+b)^*a$

7) " " Containing 'a' $\rightarrow$ ⑦ $(a+b)^*a(a+b)^*$

8) " " starting and end with different symbol. $\rightarrow$ ⑧ $(a(a+b)^*b + b(a+b)^*a)$

Which two of the following four regular expressions are equivalent?

- (i) $(00)^*(\epsilon + 0)$ $\rightarrow \epsilon, 00, 000, 0000, \dots \rightarrow$ all no $\leftarrow$
- (ii) $(00)^*$ $\rightarrow \epsilon, 00, 0000, 000000, \dots \rightarrow$ even no. $\leftarrow$
- (iii) 0^* $\rightarrow \epsilon, 0, 00, 000, 0000, \dots \rightarrow$ all no. $\leftarrow$
- (iv) $0(00)^*$ $\rightarrow \epsilon, 0, 00, 000, 0000, \dots \rightarrow$ odd no. $\leftarrow$

(a) (i) and (ii)

(b) (ii) and (iii)

☒ (c) (i) and (iii)

(d) (iii) and (iv)

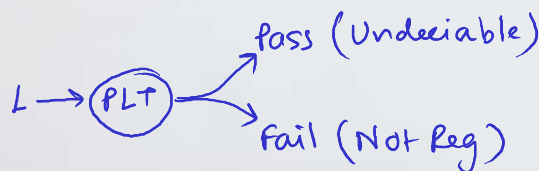
Correct

Pumping Lemma =

$\rightarrow$ check whether the language is Regular or not.
(-ve test)

Pumping Lemma - If L is an infinite language then there exists some Positive integer 'n' (Pumping length) such that any string $w \in L$ has length greater than equal to 'n' i.e. $|w| \geq n$ then w can be divided into three parts, $w = xyz$. satisfy following conditions

- { i) for each $i \geq 0$, $xy^iz \in L$
ii) $|y| > 0$ and
iii) $|xy| \leq n$



$$L = a^n b^{2n}, n \geq 0$$

$$w = \frac{aa}{x} \frac{bb}{y} \frac{bb}{z} \quad w \in L$$

$$\downarrow y^2$$

$$w_1 = \frac{aa}{x} \frac{bbbb}{y} \frac{bb}{z} \quad w_1 \notin L$$

$$\#a = 2 \rightarrow \times 3 \Rightarrow \underline{\underline{xa}}$$

$$\#b = 6 \rightarrow \times 3 \Rightarrow \underline{\underline{xb}}$$

$$w = \frac{aa}{x} \frac{bb}{y} \frac{bb}{z} \in L$$

$$\downarrow y^2$$

$$w_1 = \frac{a}{x} \frac{abab}{y} \frac{bbb}{z} \notin L$$

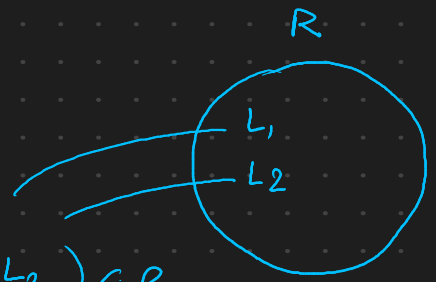
$$\#a = 3$$

$$\#b = 5$$

$\Rightarrow$ So the language is not Regular

Closure Properties of Regular Languages.

- 1) Union $(L_1 \cup L_2)$
- 2) Concatenation $(L_1 \cdot L_2)$
- 3) Closure $(*)$ L^*
- 4) Complementation $\overline{L} = \Sigma^* - L$
- 5) Intersection $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}}$
- 6) Difference $L_1 - L_2 = L_1 \cap \overline{L_2}$
- 7) Reversal $(L)^R$
- 8) Homomorphism
- 9) Reverse Homomorphism
- 10) Quotient operation
- 11) INIT
- 12) Substitution
- 13) Infinite Union $\times \times$



$$(L_1 \cup L_2) \in R.$$

$$(L_1 \cdot L_2) \in R$$

$$(L_1 \cap L_2) \in R.$$

So on.

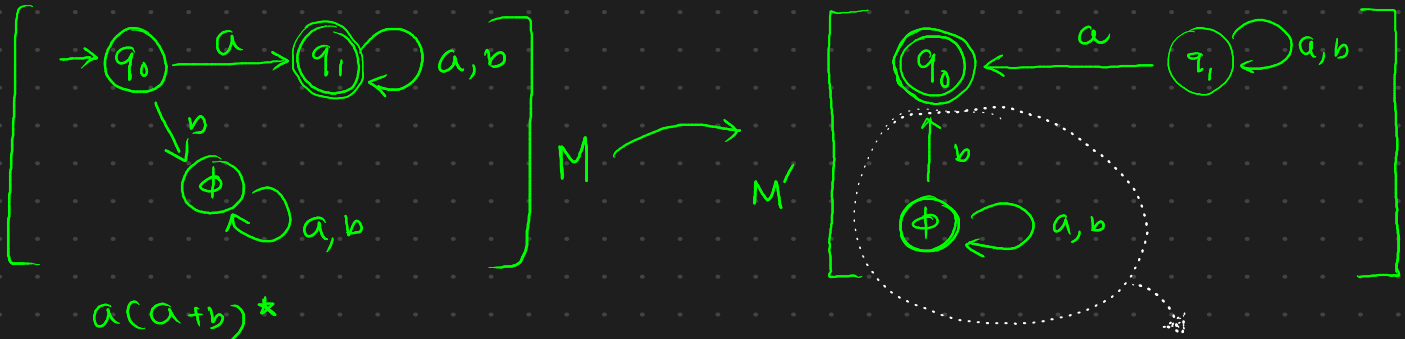
Reversal of a Language.

Regular Languages are closed under Reversal $\xrightarrow{R}$ $abc \rightarrow cba$

- 1) Make initial state of M as final state of M'
- 2) Final state of M become initial state of M'
- 3) Reverse the direction of edges of M to make M'
- 4) No change in Loop and Remove unnecessary states

Example: $L_1 = \{ \text{set of all strings over } (a,b) \text{ starting with } a \}$ $\rightarrow a(a+b)^*$

DFA



can remove
this

Quotient Operation.

Quotient operation $\begin{cases} \rightarrow \text{Left Quotient (Cut Prefix)} \\ \rightarrow \text{Right Quotient (Cut Suffix)} \end{cases}$

$$\frac{L_1}{L_2} = \{ x \mid \underline{xy} \in L, \text{ for some } y \in L_2 \} \quad \text{Right Quotient (Cutting in Right)}$$

$$\frac{L_1}{L_2} = \{ y \mid \underline{xy} \in L, \text{ for some } x \in L_2 \} \quad \text{Left Quotient (Cutting in Left)}$$

$$\begin{aligned} L_1 &= \{ 10, 100, 1010, 101110 \} \\ L_2 &= \{ 10 \} \end{aligned} \quad \left| \quad \begin{aligned} L_1 &= a^*b \\ L_2 &= ab \end{aligned} \right.$$

$$\frac{L_1}{L_2} (\text{Left}) = \frac{10}{10} \quad \frac{100}{10} \quad \frac{1010}{10} \quad \frac{101110}{10} = \{ \epsilon, 0, 10, 1110 \}$$

$$\frac{L_1}{L_2} (\text{Right}) = \frac{10}{10} \quad \frac{100}{10} \quad \frac{1010}{10} \quad \frac{101110}{10} = \{ \epsilon, \epsilon, 10, 1011 \}$$

(Not mature so ϵ)

Init Operation.

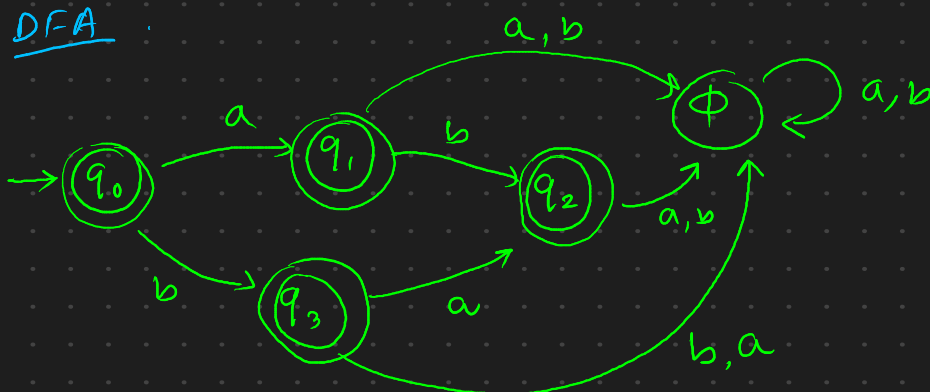
INIT operation (Initial / Prefix)
 { Set of all prefix of $w \in L$ }
 Let $L = \{ ab, ba \}$

$$L = \{ ab, ba \}$$

$$\begin{aligned} ab &= \epsilon, a, ab \\ ba &= \epsilon, b, ba \end{aligned} \quad \text{Union}$$

$$\{ \epsilon, a, b, ab, ba \}$$

DFA



→ Make all the Non final states of DFA to final to Make init.

Regular Languages are not closed under Infinite Union

$$L_1 = \{a^1b^1\}$$

$$L_2 = \{a^2b^2\}$$

$$L_3 = \{a^3b^3\}$$

Regular Language.

$$L_n = \{a^n b^n\}$$

Not Regular Language.

$$\{a^n b^n, n \geq 1\}$$

So Regular Languages are not closed under ∞ union.

Closure Properties of Various Languages

	Regular	DCFL	CFL	CSL	REC	RE
$\cup$	Yes	No	Yes	Yes	Yes	Yes
$\cap$	Yes	No	No	Yes	Yes	Yes
$\subseteq$	Yes	Yes	No	Yes	Yes	No
$\cdot$	Yes	No	Yes	Yes	Yes	Yes
$*$	Yes	No	Yes	Yes	Yes	Yes
$+$	Yes	No	Yes	Yes	Yes	Yes

DCFL = Deterministic Context Free Language

CFL = Context Free Language

CSL = Context Sensitive Language

REC = Recursive

RE = Recursive Enumerable.

$$\{a^m b^n\} \cup \{a^n b^{2n}\}$$

DCFL

DCFL

Choice $\rightarrow$ CFL $\rightarrow$ NPDA

$$\{a^m b^m c^n\} \cap \{a^n b^m c^m\}$$

CFL

CFL

$$a^m b^m c^m$$

comparisons ≥ 2

CSL